

UML BEHAVIORAL MODELING

Activity Diagrams

- *Dr. Madhavi Dave*



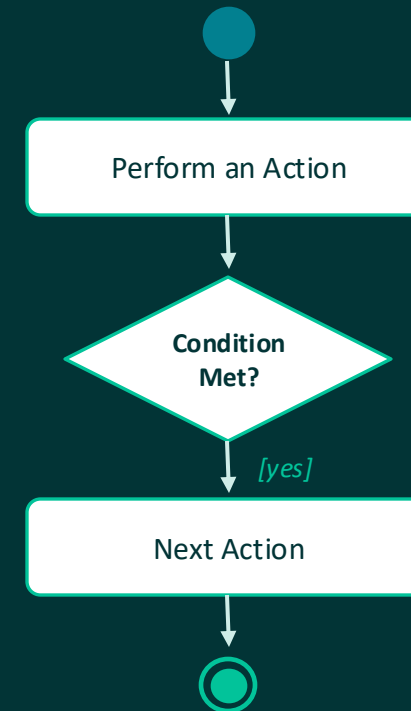
What Is an Activity Diagram?

An **activity diagram** is a UML behavioral diagram that models the **dynamic flow of activities** within a system — the sequence of actions, decisions, and conditions a process moves through from start to finish.

Roger Pressman (Software Engineering: A Practitioner's Approach) presents activity diagrams as part of **behavioral modeling** — used alongside use-case and state diagrams to represent how a system behaves as requirements are elicited and refined.

In essence: it works much like an enhanced flowchart, but adds support for **concurrency, branching, and responsibility (swimlanes)** — capabilities a plain flowchart lacks.

AT A GLANCE



*Filled circle = start · Rounded box = action ·
Diamond = decision · Ringed circle = end*

Why Activity Diagrams Matter in Software Engineering

1

Visualize Workflow

Lays out the step-by-step sequence of a process or use case so analysts, developers, and clients share one clear picture.

2

Reveal Decision Logic

Makes branching conditions and guard-based paths explicit, exposing edge cases early in requirements analysis.

3

Model Concurrency

Fork and join bars show activities that happen in parallel — something a basic flowchart cannot express.

4

Assign Responsibility

Swimlanes map each activity to the class, actor, or subsystem accountable for it, clarifying interfaces between parts.

Note

The activity diagram as an extension of the use-case, providing a graphical view of a business process, a use case, or the internal logic of a complex operation. It complements class and state diagrams by focusing purely on the flow of control and data — not on structure.

Core Symbols Used in an Activity Diagram



Initial Node

A solid filled circle marks the single starting point of the workflow.



Action State

A rounded rectangle describing one task or step performed in the process.



Decision / Branch

A diamond that splits flow into alternative paths, each labeled with a [guard condition].



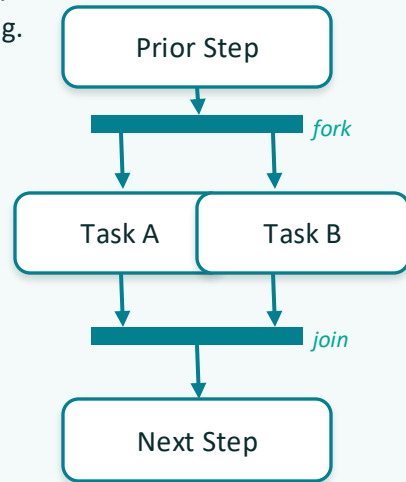
Final Node

A circle with an inner filled ring marks where the activity flow terminates.

Concurrency & Responsibility

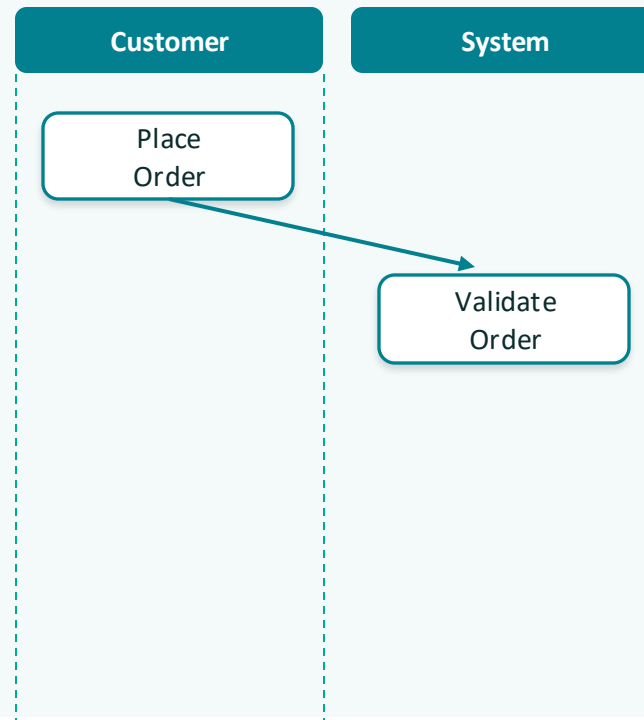
Fork & Join

A fork bar splits one flow into concurrent paths; a join bar synchronizes them back into one before continuing.



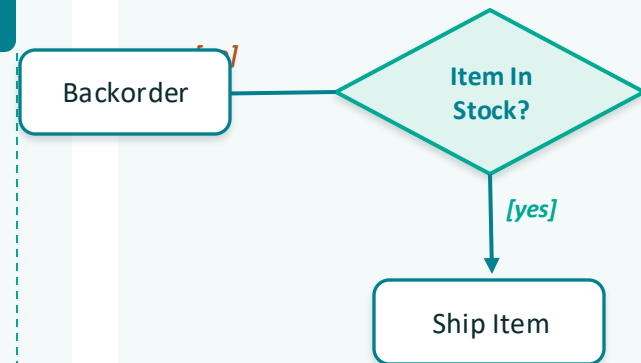
Swimlanes

Vertical or horizontal partitions assign each activity to the actor, class, or subsystem responsible for it.



Guard Conditions

A bracketed expression such as [in stock] labels an outgoing transition and must evaluate true for that path to be taken.



Rules for Constructing an Activity Diagram

1

Single Initial Node

Every diagram begins with exactly one filled-circle initial node — the unambiguous entry point of the flow.

2

Purposeful Actions

Each action state must represent one clear, atomic step; avoid vague or compound activities in a single box.

3

Directed, Unbroken Flow

Transitions are directed arrows only; every action must have at least one incoming and one outgoing path (except start/end).

4

Mutually Exclusive Guards

Each branch leaving a decision diamond carries a distinct [guard condition]; guards at one decision must not overlap.

5

Balanced Fork/Join

A fork that splits flow into N concurrent paths should be matched by a join recombining those same N paths.

6

Lanes Show Ownership

When responsibility matters, partition the diagram into swimlanes — one per actor, class, or subsystem.

7

Converge Before Exit

All branches must eventually reconverge or lead independently to a final node — no dangling, unterminated paths.

8

One Direction of Reading

Keep flow consistently top-to-bottom or left-to-right, and minimize crossing lines for readability.

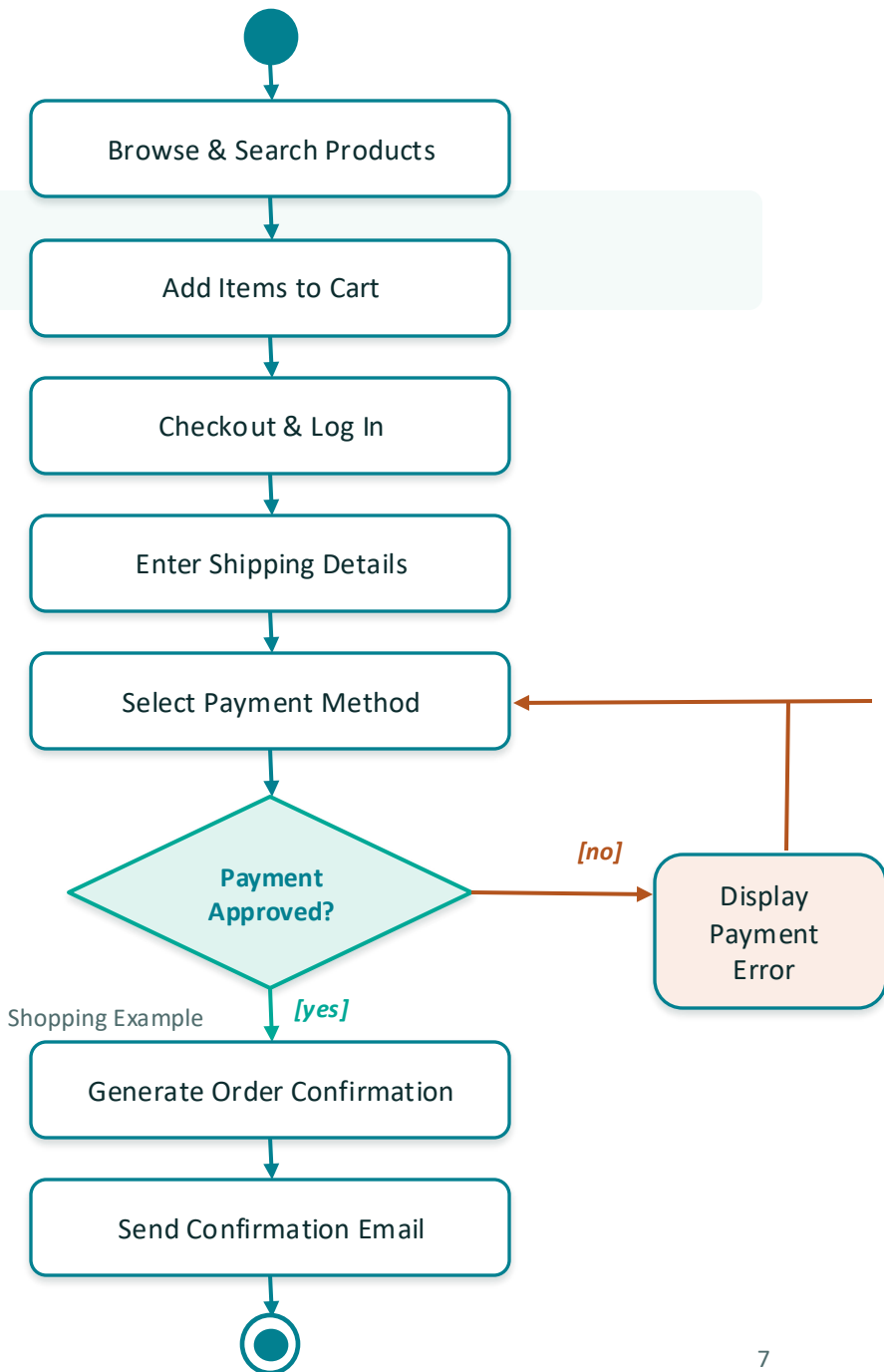
Activity Diagram — Online Shopping Application

Models the customer journey from browsing products through checkout, payment validation, and order confirmation.

READING THE FLOW

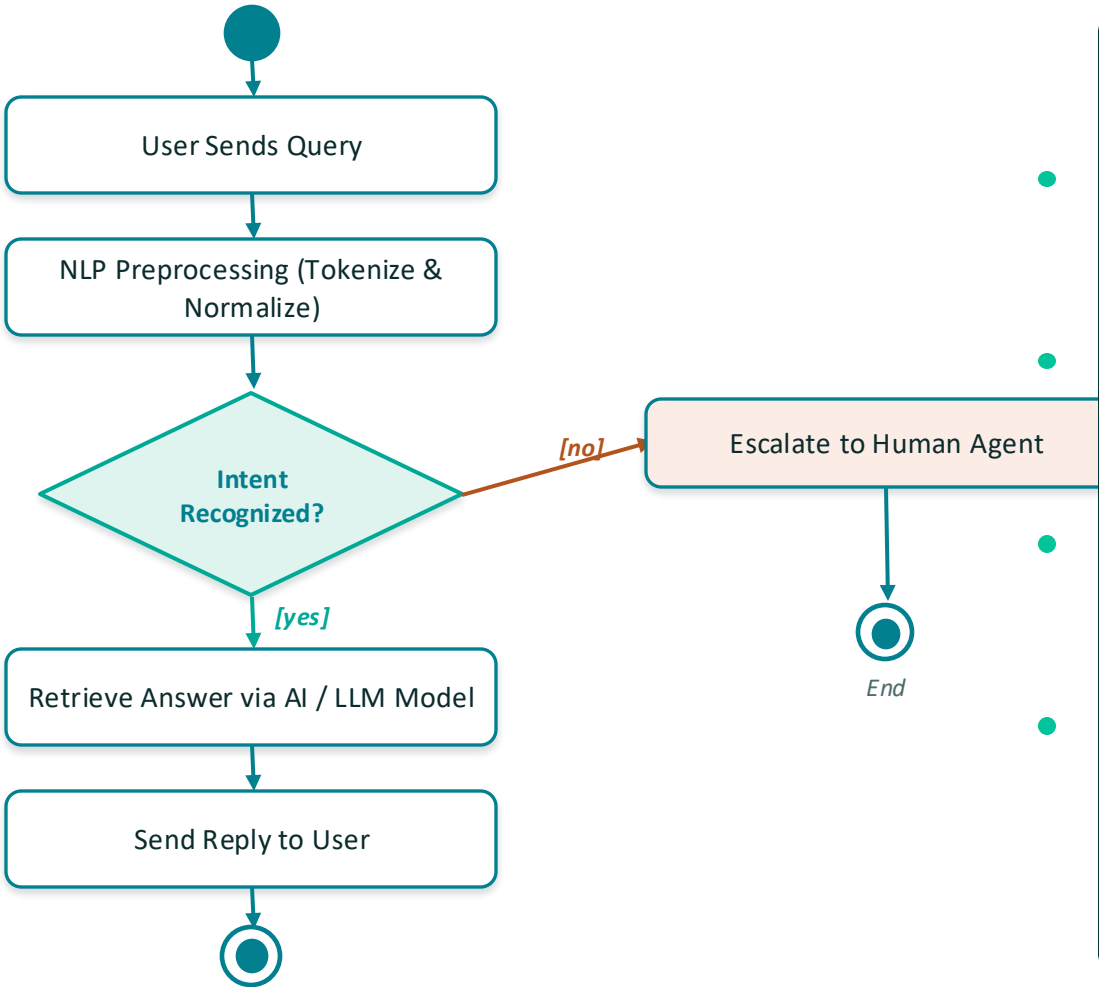
- Linear happy path: browsing through to shipping details runs top-to-bottom with no branching.
- Single decision point: “Payment Approved?” is the only guarded branch in this process.
- Error recovery loop: a declined payment routes back to “Select Payment Method” rather than dead-ending.
- Convergence: both the retry loop and the approved path eventually reach the single final node.
- Swimlanes (Customer vs. System) could further split this diagram by who performs each step.

Activity Diagrams — Online Shopping Example



Activity Diagram — AI Chatbot Query Handling

Traces how a conversational AI assistant processes an incoming user query, from intent recognition through to a generated response.



WHY THIS FITS AN ACTIVITY DIAGRAM

The decision node captures a real AI uncertainty point — confidence in intent classification.

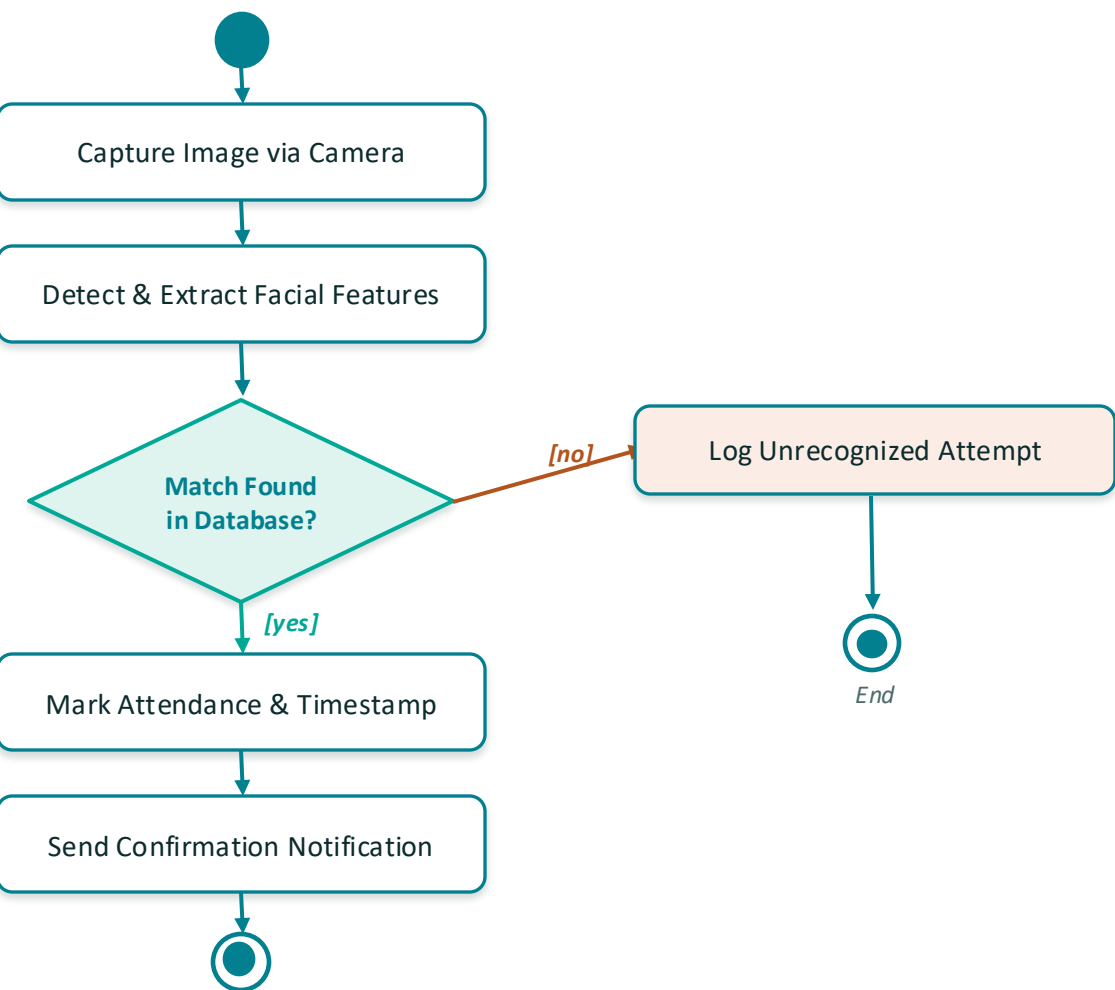
The [no] path models graceful failure: handing off to a human rather than looping forever.

The [yes] path shows the model-driven core: retrieval or generation, then delivery.

Concurrency (fork/join) could extend this to run intent detection and sentiment analysis in parallel.

Activity Diagram — Face Recognition Attendance System

Shows the vision pipeline from image capture to feature matching, ending in either a logged attendance record or an unrecognized-attempt log.

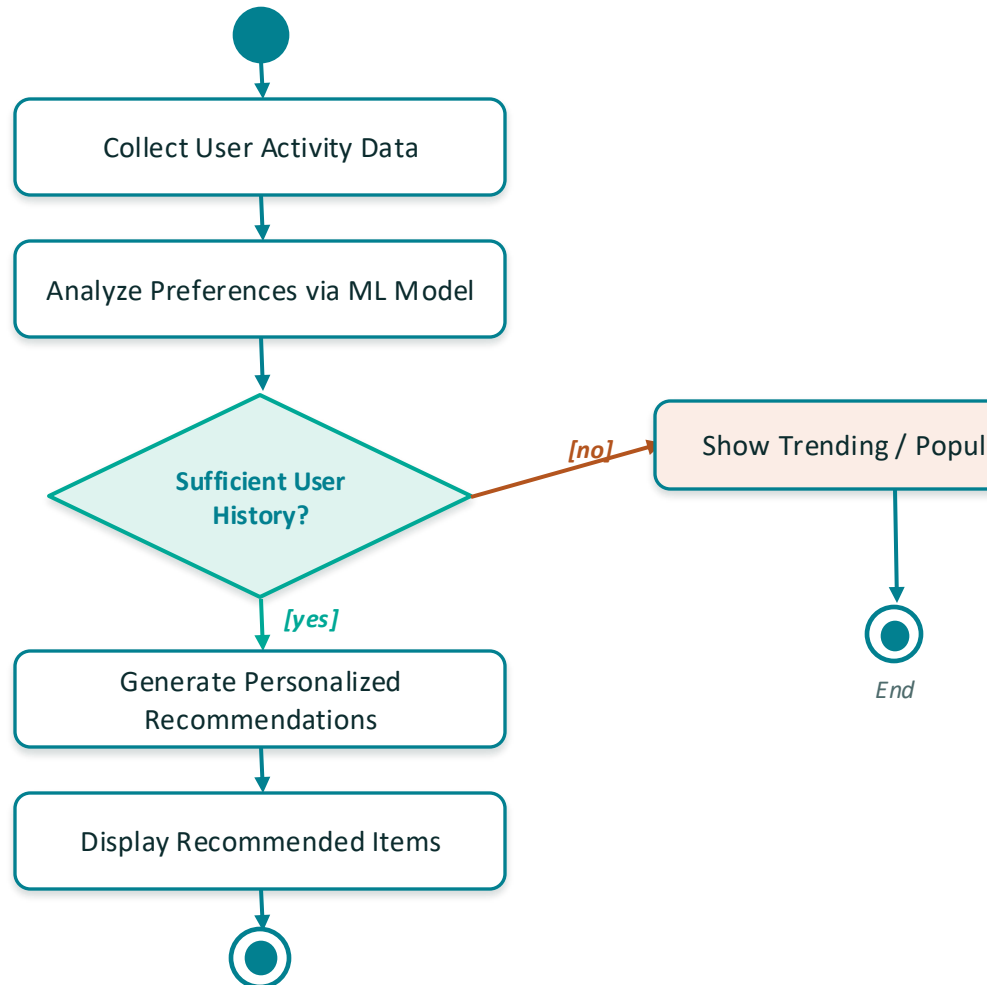


WHY THIS FITS AN ACTIVITY DIAGRAM

- Feature extraction and database matching are atomic, well-scoped action states.
- The guard condition mirrors the model's classification threshold in plain language.
- Both outcomes converge conceptually on the system being auditable — every attempt is logged.
- A swimlane split (Camera / AI Engine / Attendance Server) would clarify component ownership.

Activity Diagram — AI-Based Recommendation Engine

Illustrates how a recommender (e.g. streaming or e-commerce) branches between personalized results and a cold-start fallback.



WHY THIS FITS AN ACTIVITY DIAGRAM

- The decision node encodes the classic “cold-start” problem in recommender systems.
- The [no] branch is a sensible default, not an error — activity diagrams model any valid path.
- The [yes] branch reflects the personalization pipeline the ML model actually performs.
- Both branches converge on the same user-facing outcome: a populated results screen.

SUMMARY

Key Takeaways

- 1 Activity diagrams model dynamic flow — actions, decisions, concurrency, and ownership — as part of Pressman's behavioral modeling.
- 2 Consistent notation (initial/final nodes, actions, diamonds, fork/join, swimlanes) keeps diagrams readable across teams.
- 3 The eight construction rules ensure every path starts cleanly, branches unambiguously, and converges to a defined end.
- 4 The online shopping example shows a real-world linear-with-recovery-loop process end to end.
- 5 The three AI examples confirm the notation scales naturally to model uncertainty in intelligent systems — intent confidence, match confidence, and data sufficiency.